



Aequs Special Economic Zone, Hattargi, Belagavi - 591243, Karnataka, INDIA.

[IT DEPARTMENT]

Project Report on

**Full Stack Web Development: Tool Code Library
System**

submitted

in fulfillment of the requirements for the completion of

[Internship Program]

IN

[COMPUTER SCIENCE AND ENGINEERING]

Submitted By

[Amey Nitin Pawar]

Under the guidance of

[Akash Genji]

[Lead Engineer]

Abstract

This report presents the design and implementation of a Web-Based Tool Code Library System developed to standardise the creation, approval, and lifecycle management of unique industrial tool code identifiers within a manufacturing organisation.

Utilising a full-stack approach with the Python Django framework, the application features a robust backend architecture paired with a responsive, user-friendly frontend interface. Core functionalities include a dynamic Role-Based Access Control (RBAC) model driven entirely by a database-managed Role model with seven configurable boolean permission flags, a two-stage approval workflow (Draft → Pending → Approved/Rejected), and an automatic Tool Code generation algorithm that encodes tool specifications into a structured, collision-safe alphanumeric string of up to 40 characters.

A comprehensive Soft Delete pattern, implemented via an abstract `SoftDeleteModel` base class and a custom `ActiveManager`, ensures that no master data record (Supplier, Raw Material, Joint Type, or Field) is ever permanently deleted, thereby preserving historical data integrity across all approved records.

An `AuditLog` model records every significant system event—such as login, blocked login, and failed login—along with the actor’s identity, IP address, HTTP method, and timestamp.

By digitising and centralising the tool code governance workflow, this system eliminates manual duplication errors, enforces consistent formatting, provides a complete audit trail, and significantly improves operational accuracy.

Ultimately, this project delivers a scalable digital solution tailored for modern manufacturing and procurement environments.

Keywords: Full-Stack Development, Django Framework, Tool Code Management, Role-Based Access Control, Soft Delete, Audit Log.

Acknowledgement

I would like to express my sincere gratitude to my company guide, **Mr.Akash Genji**, for their invaluable mentorship and guidance throughout my internship at Aequus SEZ Private Limited. Their technical expertise, constructive feedback, and continuous support were instrumental in shaping my understanding of real-world Django application development and ensuring the successful completion of this project.

I am also thankful to the entire **Aequus SEZ IT team** for providing a collaborative and motivating work environment. The exposure to the company's structured engineering processes and high professional standards has significantly enhanced both my technical and interpersonal skills. Contributing to a real-world software project within such an innovative culture has been an enriching experience that bridged the gap between academic theory and industry practice.

Finally, I extend my sincere appreciation to my college faculty at **Arvind Gavali College of Engineering** for their foundational academic guidance. Their ongoing support equipped me with the necessary theoretical knowledge to confidently tackle and solve real-world industry challenges.

[Amey Nitin Pawar]

Contents

Abstract	i
Acknowledgement	ii
1 Introduction	1
1.1 Overview	1
1.2 Project Objectives	2
2 Methodology	3
2.1 Development Methodology	3
2.2 System Architecture: The MVT Pattern	3
2.3 Phases of Development	4
2.4 Technology Stack	4
2.5 System Requirements	5
2.5.1 Hardware Requirements	5
2.5.2 Software Requirements	5
3 System Design	6
3.1 System Architecture Overview	6
3.2 Database Schema and Relational Structure	6
3.3 Core Modules and System Logic	7
3.3.1 1. Authentication and Audit Module	7
3.3.2 2. Role-Based Access Control (RBAC) Module	7
3.3.3 3. Dashboard and Analytics Module	8
3.3.4 4. Tool Code Generation Engine	8
3.3.5 5. Dynamic Form Configuration Module	8
3.3.6 6. Approval Workflow Module	9
3.3.7 7. Master Data Management Module	9
3.3.8 8. Reporting and Search Module	9
4 Implementation	10
4.1 Implementation Overview	10
4.2 Data Validation and Security Measures	10
4.3 Backend Logic and AJAX Architecture	11
4.4 Core Interfaces and System Navigation	11
4.4.1 1. User Authentication and Security	11
4.4.2 2. The Main Dashboard	12
4.4.3 3. Tool Code Request Creation and Editing	12

4.4.4	4. Approval Review Interface	14
4.4.5	5. Master Data Management Console	15
4.4.6	6. Role Master and User Management	17
4.4.7	7. Master Reporting	18
5	Conclusion	20
5.1	Conclusion	20
5.2	Future Scope and Enhancements	21
	References	22

List of Figures

4.1	Secure User Authentication Interface	11
4.2	Main Dashboard displaying metrics, charts, and the active task list	12
4.3	Tool Code Creation Form — dynamic sub-fields rendered after Primary Description selection	13
4.4	Edit Modal showing the red Rejection Reason banner for a returned request	14
4.5	Approval Review Modal with read-only specification and action buttons . .	15
4.6	Master Data Table showing Active and Inactive rows with toggle controls .	16
4.7	Field Attributes Master with the Dropdown Option Builder interface . . .	16
4.8	Role Master Interface with granular boolean permission toggle switches . .	17
4.9	User Creation Interface	18
4.10	Master Report Interface with date-range filters and real-time search	19

Chapter 1

Introduction

1.1 Overview

In today’s fast-paced manufacturing and procurement environment, professionals and organisations must manage numerous tool specifications, supplier relationships, and material catalogues simultaneously. Assigning tool code identifiers manually frequently leads to duplicate codes, non-standard formats, overlooked approvals, and a complete absence of audit traceability—all of which introduce costly procurement errors and operational inefficiencies.

Traditional methods, such as spreadsheets or informal email-based approvals, lack the structure and automation required for reliable tool code governance. There is no mechanism to prevent two engineers from independently assigning the same code to different specifications, no formal validation step, and no record of who created or approved a code and when.

The Tool Code Library System developed in this project directly addresses these challenges. It enables users to seamlessly create, categorise, and submit tool code requests through a dynamic, attribute-driven form. Approvers review and validate submissions through a structured workflow before a unique, system-generated code is permanently assigned. Administrators maintain all reference data (Suppliers, Raw Materials, Joint Types, and dynamic Field Attributes) through a governed master data module that ensures historical records remain intact.

Under the hood, the application is built on a scalable and modular Django architecture. Core components—authentication with audit logging, dynamic RBAC, tool code generation, workflow state management, and reporting—are developed as independent modules to simplify maintenance, testing, and future enhancements. Strict input validation and a soft delete pattern prevent both data entry errors and irreversible data loss.

Ultimately, this project demonstrates how digitising and automating tool code governance significantly reduces manual effort, enforces standardisation, and improves overall operational accuracy.

1.2 Project Objectives

The primary objectives of this project are:

- **Dynamic Role-Based Access Control:** To implement a fully database-driven RBAC system using a Role model with seven configurable boolean permission flags, enabling Super Administrators to define access policies at runtime without requiring code changes.
- **Structured Approval Workflow:** To build a two-stage workflow (Draft → Pending → Approved/Rejected) with mandatory rejection reasons, ensuring that every tool code is validated by an authorised approver before activation.
- **Automated Code Generation:** To develop a deterministic tool code generation algorithm that encodes tool type, fixed values, dynamic attributes, joint type, and material into a structured string with a strict 40-character limit and a collision-safe uniqueness mechanism.
- **Master Data Integrity:** To manage all reference data through a Soft Delete pattern, ensuring that deactivating a Supplier, Material, or Joint Type does not affect historical approved records.
- **User Experience (UX):** To design a user-friendly and dynamic interface where form fields adapt automatically based on the selected tool type, and role-specific dashboards display only contextually relevant data.

Chapter 2

Methodology and System Requirements

2.1 Development Methodology

To ensure a structured yet flexible development process, this project adopted the **Agile Software Development Lifecycle (SDLC)**. Unlike traditional rigid models, Agile allows for iterative development, continuous testing, and the ability to adapt to stakeholder feedback throughout the internship.

The project was divided into manageable development sprints. The first sprint focused on core infrastructure: the `SoftDeleteModel` abstract base, the dynamic Role model, user authentication with `AuditLog`, and the basic `ToolRequest` workflow. Subsequent sprints progressively introduced the dynamic attribute engine, the tool code generation algorithm, master data modules, and the reporting dashboard. This iterative approach ensured that a working prototype was consistently available for review and refinement based on mentor guidance during the internship at Aequus.

2.2 System Architecture: The MVT Pattern

The application was built using a full-stack approach, specifically leveraging the **Model-View-Template (MVT)** software design pattern provided by the Python Django framework. This architecture promotes a clear separation of concerns, making the codebase easier to maintain and scale:

- **Model (Data Access Layer):** Acts as the interface between the application and the database. It defines the structure of all data—role permissions, tool requests, master data records, and audit events—ensuring integrity through Django’s Object-Relational Mapper (ORM). The application defines 12 Django models, several of which inherit from the abstract `SoftDeleteModel` base.
- **View (Business Logic Layer):** Serves as the bridge between the Model and the Template. It processes all user requests, enforces role-based permission checks, executes the tool code generation algorithm, handles AJAX calls for dynamic form rendering, and manages workflow state transitions. The application contains 25 view functions.

- **Template (Presentation Layer):** Handles the frontend user interface through 14 HTML templates. Dynamic form fields are rendered client-side using JavaScript based on JSON responses from AJAX endpoints, providing an interactive experience without full page reloads.

2.3 Phases of Development

The execution of the project was divided into four primary phases:

1. **Requirement Analysis:** Understanding the core problem of manual tool code duplication and inconsistency; defining the scope, including RBAC, approval workflow, soft delete, and audit logging; and identifying all master data entities along with their dynamic attribute requirements.
2. **System Design:** Designing the 12-model database schema with all necessary relationships; mapping the complete URL structure (29 routes); designing the tool code generation algorithm and workflow state machine; and creating wireframes for all 14 templates.
3. **Implementation:** Developing the backend logic using Python and Django for all 25 view functions; setting up the relational database; creating all 14 HTML templates with dynamic JavaScript; and integrating AJAX endpoints for real-time form behaviour.
4. **Testing and Refinement:** Conducting functional testing across 24 test cases covering workflows, permission boundaries, soft delete behaviour, and code generation edge cases. Refining the user interface based on user acceptance testing feedback.

2.4 Technology Stack

Selecting the appropriate tools was essential for building a robust and modern web application. The following technology stack was used:

- **Backend Development:** Python 3.x and the Django Web Framework.
- **Frontend Development:** HTML5, CSS3, and JavaScript (following responsive web design principles).
- **Database:** SQLite was used during development, with the system structured to support easy migration to a production database such as Microsoft SQL Server.
- **Version Control:** Git and GitHub were used to track code changes and maintain version history.

2.5 System Requirements

To ensure optimal performance and accessibility, the following system requirements were defined:

2.5.1 Hardware Requirements

The hardware specifications required to run the system are minimal, ensuring usability across standard corporate devices:

- **Processor:** Minimum Intel Core i3 (or equivalent AMD processor)
- **RAM:** 4 GB (8 GB recommended for multitasking and backend execution)
- **Storage:** Minimum 10 GB of available disk space
- **Display:** Standard monitor with at least 1024×768 resolution

2.5.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Web Browser:** Modern browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge (with JavaScript enabled)
- **Development Environment:** Visual Studio Code (VS Code) or PyCharm IDE

Chapter 3

System Design and Architecture

3.1 System Architecture Overview

The Tool Code Library System is built on a standard **three-tier web architecture**. By utilizing Django's Model-View-Template (MVT) pattern, the system is divided into three distinct layers that communicate through structured interfaces. This layered approach ensures scalability, maintainability, and efficient handling of dynamic workflows such as tool code generation and approval processing.

- **Presentation Layer (Frontend):** Built using HTML, CSS, Bootstrap, and JavaScript, this layer provides a responsive and role-adaptive user interface. Navigation menus and UI components are dynamically rendered based on user permissions. AJAX endpoints enable real-time updates such as dynamic form loading without requiring full page reloads.
- **Application Layer (Backend):** Developed using Python and Django, this layer implements the core business logic. It manages authentication, enforces role-based access control (RBAC), executes tool code generation algorithms, and controls the approval workflow. All critical operations are validated at this layer before interacting with the database.
- **Database Layer:** Microsoft SQL Server is used as the centralized data store. It maintains all relational data including users, tool requests, attributes, and master data. The database ensures data integrity, efficient querying, and long-term persistence.

3.2 Database Schema and Relational Structure

The database schema is designed using a highly relational approach to maintain data consistency and scalability. It ensures that all tool code configurations, requests, and master data remain structured and traceable.

The system is organized into the following logical groups:

- **User and Security Models:** Includes User, UserProfile, Role, and AuditLog. These models manage authentication, permissions, and system activity tracking.

- **Dynamic Configuration Models:** Includes Field, FieldAttribute, FieldAttributeOption, and FieldFixedValue. These define the dynamic structure of tool codes and form inputs.
- **Master Data Models:** Includes Supplier, RawMaterial, and JointType. These provide standardized data used in tool classification and code generation.
- **Core Transaction Models:** Includes ToolRequest and ToolRequestAttribute. These store tool requests and their dynamic attribute values.

The relational structure ensures:

- Data normalization and minimal redundancy
- Strong referential integrity via foreign keys
- Support for dynamic schema configuration

3.3 Core Modules and System Logic

To ensure smooth system operation and maintain modular design, the application is divided into the following core modules:

3.3.1 1. Authentication and Audit Module

This module secures system access and tracks user activities.

- **User Authentication:** Users log in using secure Django authentication mechanisms.
- **Audit Logging:** Every important action such as login attempts, approvals, and errors is recorded with timestamp, user details, and IP address.
- **Access Protection:** All protected views require authentication and permission validation before execution.

3.3.2 2. Role-Based Access Control (RBAC) Module

The RBAC module provides dynamic permission control across the system.

- **Permission-Based Access:** Each role defines access to features such as tool codes, approvals, master data, and reports.
- **Dynamic Role Assignment:** Administrators can assign or modify roles without changing the codebase.
- **Security Enforcement:** Unauthorized users are restricted from accessing sensitive modules.

3.3.3 3. Dashboard and Analytics Module

The Dashboard serves as the primary interface for monitoring system activity.

- **Summary Metrics:** Displays Total Requests, Pending Approvals, Approved Tools, and Rejected Requests.
- **Visual Analytics:** Includes charts such as status distribution and category-based analysis.
- **Recent Activity:** Displays recently created tool requests for quick access and tracking.

3.3.4 4. Tool Code Generation Engine

The generation algorithm, implemented as `_build_code_from_values()` on the `ToolRequest` model, encodes tool specifications into a structured string:

SHORT_CODE - FIXED - attr1 X attr2 - attr3 - J - MA (≤ 40 chars)

Encoding rules:

- **Text attributes** — truncated to 2 uppercase characters.
- **Numeric attributes** — kept as full value (e.g., 14, 1.5).
- **40-character guard** — attributes stop being added when the next addition would exceed 37 characters (reserving 3 characters for the uniqueness counter suffix -01).
- **Collision safety** — if the base code already exists in the database, -01, -02, etc. are appended until a unique code is found.

3.3.5 5. Dynamic Form Configuration Module

This module enables flexible and configurable input forms.

- **Attribute Definition:** Administrators define attributes such as text, numeric, and dropdown fields.
- **Dropdown Builder:** Allows dynamic creation and management of selectable options.
- **Field Ordering:** Controls the sequence and validation of attributes during tool code creation.

3.3.6 6. Approval Workflow Module

This module manages the lifecycle of tool requests.

- **Request Lifecycle:** Requests progress through Draft → Pending → Approved or Rejected stages.
- **Approval Actions:** Authorized users can approve or reject requests with comments.
- **Audit Tracking:** Tracks who created, reviewed, and modified each request.

3.3.7 7. Master Data Management Module

This module manages all static data required for tool code generation.

- **Supplier Master:** Stores supplier details and ordering codes.
- **Raw Material Master:** Maintains material classifications.
- **Joint Type Master:** Defines tool connection types.
- **Field Master:** Defines tool categories and base code structures.

3.3.8 8. Reporting and Search Module

Provides advanced data retrieval and analysis features.

- **Filtering:** Allows filtering of tool requests by date and status.
- **Search Functionality:** Enables quick lookup of tool codes.
- **Pagination:** Efficiently handles large datasets for better performance.

Chapter 4

Implementation and User Manual

4.1 Implementation Overview

The implementation phase is where the system design was brought to life. For the Tool Code Library System, this meant translating the Django architecture into a working, production-ready application. A major focus was ensuring that the permission system, workflow logic, and code generation algorithm operated reliably and securely under all input conditions — including edge cases such as identical supplier selections, missing required attributes, and simultaneous code generation for identical specifications.

4.2 Data Validation and Security Measures

Throughout the implementation, a strong emphasis was placed on making the system fail-safe. Multiple layers of validation are applied at both the client and server level:

- **Supplier Uniqueness:** The backend explicitly checks `supplier1_id == supplier2_id` before creating a `ToolRequest`, returning a validation error if they match.
- **Required Field Enforcement:** On submission (not draft save), the view iterates through all `FieldAttribute` records with `is_required=True` for the selected tool type and verifies each has a non-empty POST value before proceeding.
- **Role Permission Guards:** Every sensitive view begins by reading the current user's `Role` boolean flags. Any mismatch redirects to the dashboard — no feature content is ever rendered to an unauthorised user.
- **Orphan Account Prevention:** The `user_creation` view deletes the newly created `User` object if an invalid `role_id` is submitted (e.g., via form tampering), preventing accounts from existing without a role assignment.
- **Super Admin Protection:** The `role_master` view explicitly checks the role name before allowing a toggle action, permanently blocking deactivation of the `Super Admin` role.

4.3 Backend Logic and AJAX Architecture

A significant technical feature is the dynamic form rendering system, which provides real-time field adaptation without page reloads. Three JSON endpoints serve the frontend:

- **get_field_attributes:** Returns all `FieldAttribute` records (with options for `select-type` fields) for a given `field_id`. Called when the Primary Description dropdown changes; triggers JavaScript to inject the appropriate input elements into the DOM.
- **get_fixed_values:** Returns `FieldFixedValue` records including `is_deleted` status, allowing the frontend to visually grey-out inactive values while still displaying them.
- **get_request_data:** Returns all fields and attributes of an existing `ToolRequest` as JSON, used to pre-populate the Edit modal without a separate page load.

4.4 Core Interfaces and System Navigation

With the backend automation running securely, the frontend was developed to facilitate seamless user interaction. The following sections outline the core workflows.

4.4.1 1. User Authentication and Security

Security is enforced via a strict login protocol. Users authenticate using assigned credentials. The Django backend evaluates the `Role` object attached to their `UserProfile`, checks whether the role is active (`is_deleted=False`), and writes a corresponding `AuditLog` entry before either granting access or returning an error.

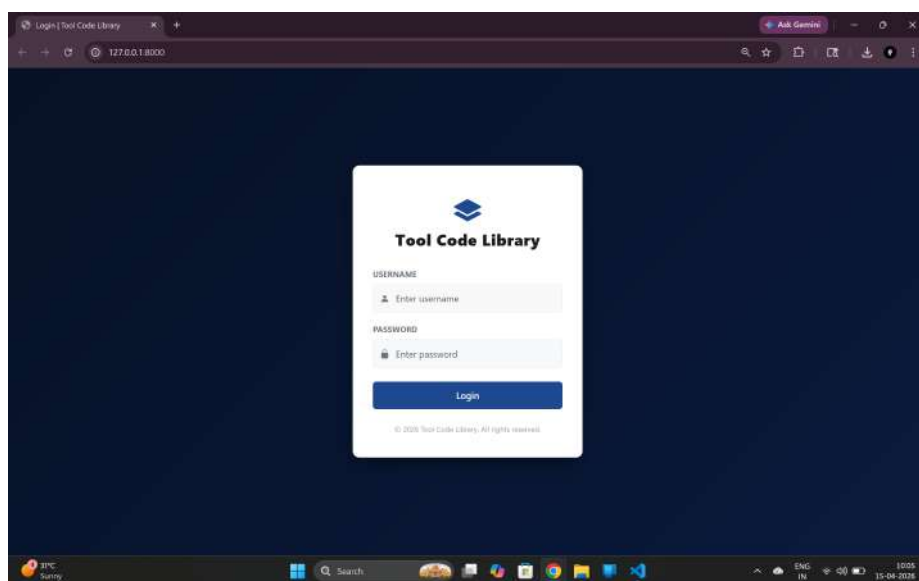


Figure 4.1: Secure User Authentication Interface

4.4.2 2. The Main Dashboard

The dashboard provides immediate, role-specific clarity on the user's workload.

- A **summary bar** shows Total Requests, Pending Approval, Approved Tools, and Rejected Requests counts — scoped to the user's own records (Requesters) or the full system (Approvers/Admins).
- **Visual charts** — a Status Distribution donut graph and a Requests by Tool Category bar graph — are rendered via Chart.js using JSON data passed from the view context.
- An **admin stats panel** (visible only to users with `can_access_master`) shows total users, suppliers, materials, and field types.
- An **approver focus panel** (visible only to users with `can_approve_requests`) highlights the number of urgent pending requests with a direct link to the tool code queue.

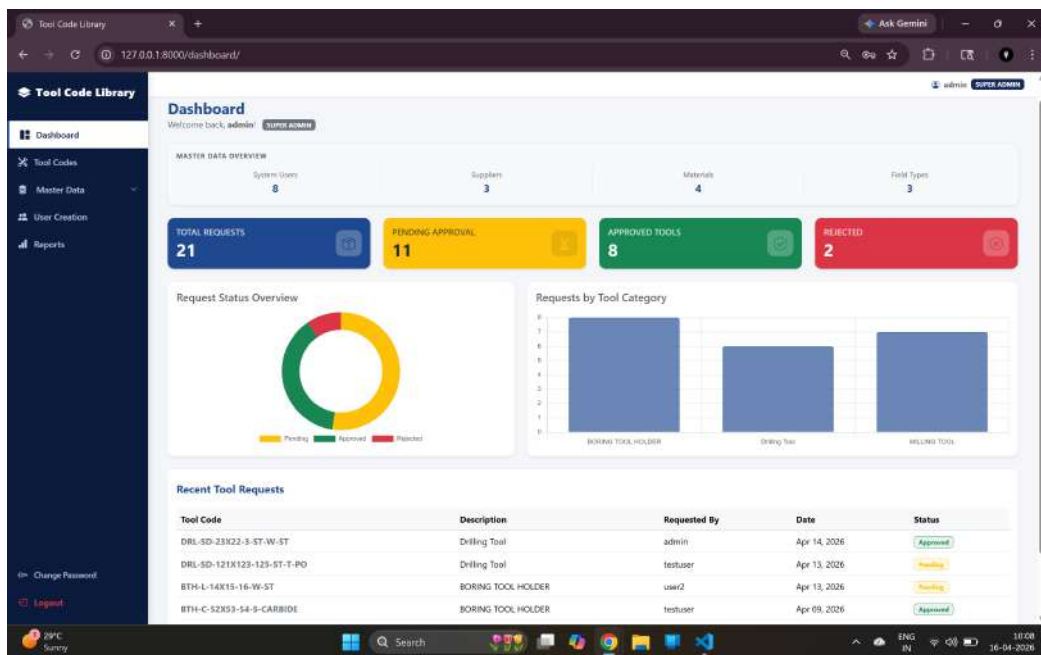


Figure 4.2: Main Dashboard displaying metrics, charts, and the active task list

4.4.3 3. Tool Code Request Creation and Editing

The tool code creation interface handles the full request lifecycle through a single unified view (`/tool-code/`), distinguished by a `form_action` POST parameter.

- **Create** (`form_action='create'`): The Requester selects a **Primary Description**. The form **dynamically expands** via an AJAX call to `/get-field-attributes/` to

show the tool-specific sub-fields for that type (e.g., Flute Length, Bore Size, Coating). The user selects Supplier 1, Supplier 2, Raw Material, and Joint Type from dropdowns populated by active master data records. Clicking **Save as Draft** stores the request with `status='Draft'` and `tool_code='[DRAFT]'` — bypassing strict validation. Clicking **Submit Request** runs full validation and, on success, calls `generate_tool_code()` to produce the final unique code.

- **Edit (form_action='edit')**: Available only for **Draft** and **Rejected** requests owned by the current user. The Edit modal is pre-populated via `/get-request-data/`. If the request was **Rejected**, a red banner displays the Approver's exact rejection reason. Existing attribute rows are deleted and recreated from the new POST data on save.

Figure 4.3: Tool Code Creation Form — dynamic sub-fields rendered after Primary Description selection

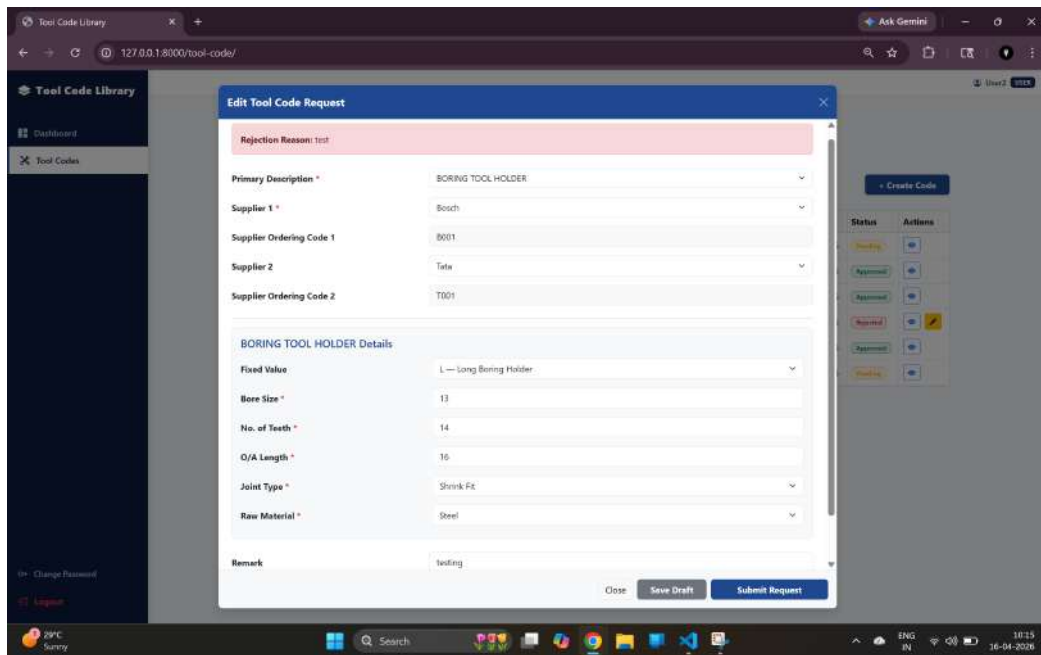


Figure 4.4: Edit Modal showing the red Rejection Reason banner for a returned request

4.4.4 4. Approval Review Interface

The approval workflow is handled through a read-only review modal accessible from the tool code list. Approvers (users with `can.approve.requests = True`) can view the full specification of any pending request across four sections: Tool Information, Specifications, Supplier Data, and User Remarks.

- **Approve:** Sets `status = 'Approved'`, records `reviewed_by = request.user` and `reviewed_at = timezone.now()`. The Tool Code is now immutable.
- **Reject:** A secondary modal requires a mandatory `reject_reason` text. The reason is stored on the `ToolRequest` and displayed to the Requester as a red banner in their next Edit session.

Status Color Legend

Color Badge	Meaning
DRAFT	Saved but incomplete; not yet submitted for review.
PENDING	Submitted and awaiting an Approver decision.
APPROVED	Finalised; Tool Code is active and immutable.
REJECTED	Returned by an Approver with a documented reason.

The screenshot shows a web application interface for reviewing a tool code request. The browser address bar shows the URL `127.0.0.1:8000/review_request/28/`. The application has a dark blue sidebar with navigation links: Dashboard, Tool Codes, Master Data, User Creation, and Reports. The main content area is titled 'Tool Code: DRL-SD-121X123-125-ST-T-PO' and includes a 'Pending Review' button. The form is divided into several sections:

- Tool Information:** Contains fields for Description (Drilling Tool), Joint Type (T Joint), Fixed Value (SD — Solid Drill), and Raw Material (Powder Metal).
- Drilling Tool Specifications:** Contains fields for Drill Diameter (121), Flute Length (121), No. of Flutes (125), and Drill Type (STEP DRILL).
- Supplier Information:** Contains fields for Supplier 1 (Bosch), Supplier 2 (SPF), Ordering Code 1 (8001), and Ordering Code 2 (5002).
- User Remarks:** A text area with the placeholder 'No remarks provided.'

At the bottom of the modal, there are three buttons: 'Approve' (green), 'Reject' (red), and 'Close' (grey). A footer note indicates 'Requested by: testuser on Apr 13, 2025'.

Figure 4.5: Approval Review Modal with read-only specification and action buttons

4.4.5 5. Master Data Management Console

A secure management console is available to users with `can_access_master = True`, accessible through the navigation menu. This console covers four master data modules, all following the same Add / Toggle pattern:

- **Supplier Master:** Administrators can add suppliers with a name and ordering code, and toggle their active/inactive status. Active suppliers appear in request dropdowns; inactive ones disappear from forms but remain in all historical approved records.
- **Raw Material Master:** Manages raw material entries with the same soft delete toggle pattern.
- **Joint Type Master:** Manages joint type classifications with the same soft delete toggle pattern.
- **Field (Primary Description) Master:** Manages the top-level tool type entries. Each field has a `short_code` that becomes the first segment of the generated Tool Code.

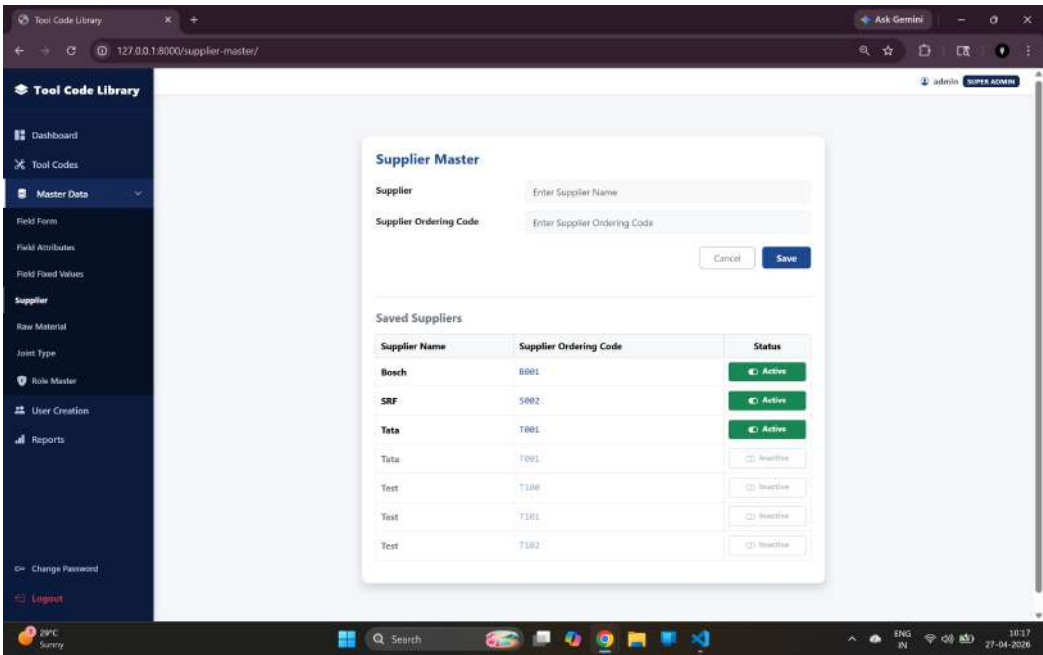


Figure 4.6: Master Data Table showing Active and Inactive rows with toggle controls

- **Field Attributes Master:** This is the most configuration-rich module. Administrators select a Field (tool type) and manage its dynamic sub-field definitions. Five actions are supported: *add* (create a new attribute, including building dropdown options for *select*-type fields), *edit* (update name, type, and required flag), *toggle* (activate/deactivate), *add_option* (append a dropdown value, auto-uppercased), and *delete_option* (soft-delete an option value).

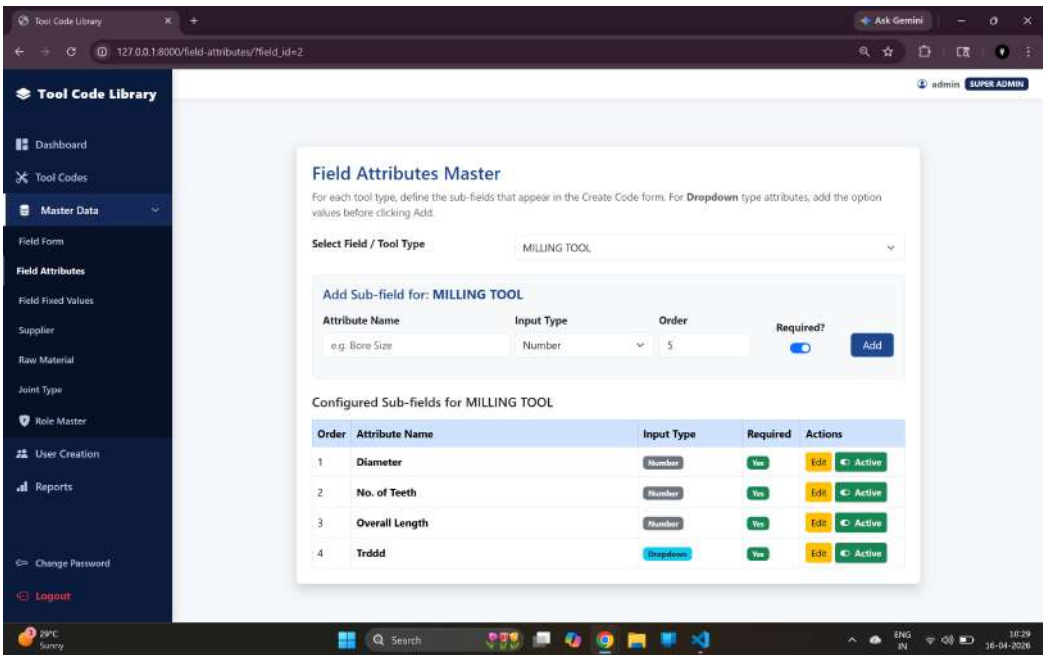


Figure 4.7: Field Attributes Master with the Dropdown Option Builder interface

4.4.6 6. Role Master and User Management

- **Role Master** (requires `can_manage_roles`): Super Administrators create new roles with any combination of the seven permission flags, edit existing roles, and toggle role status. The `Super Admin` role is permanently protected from deactivation at the view level.

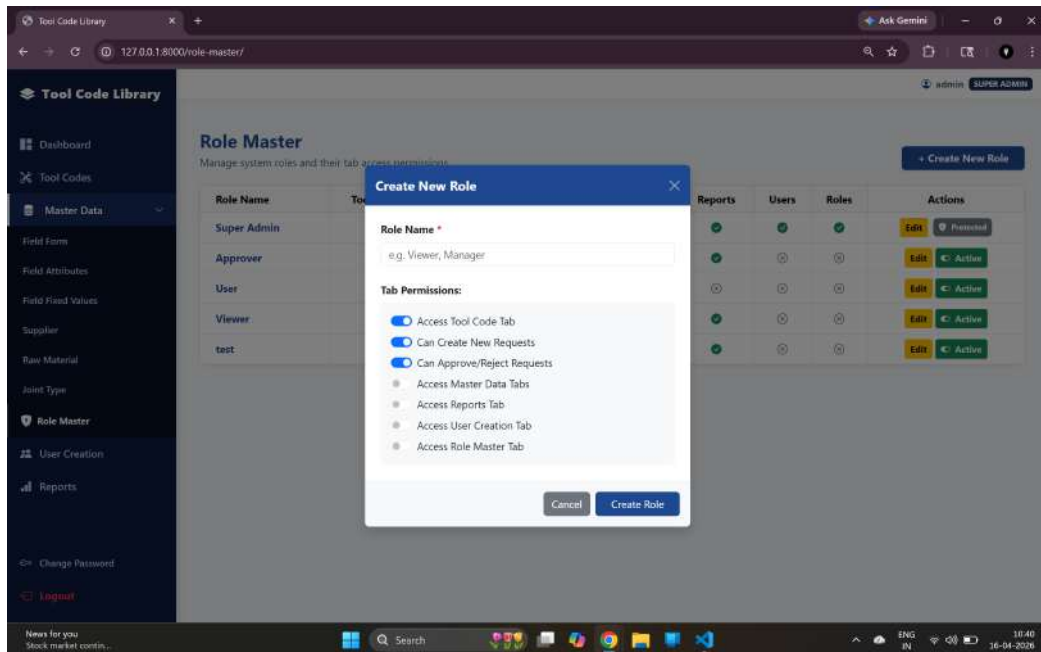


Figure 4.8: Role Master Interface with granular boolean permission toggle switches

- **User Creation** (requires `can_manage_users`): Creates a Django User and an associated UserProfile in a single operation. The form features a real-time password strength meter that strictly enforces system security policies (requiring a minimum of 8 characters, including letters, numbers, and symbols).

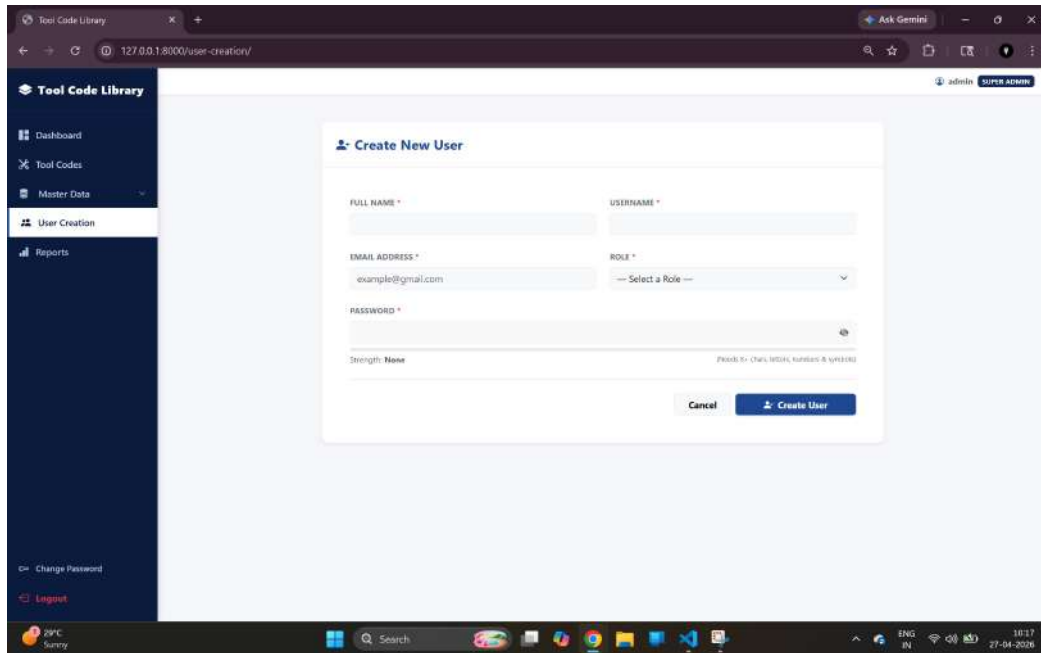


Figure 4.9: User Creation Interface

4.4.7 7. Master Reporting

The Master Report module (requires `can_access_reports`) provides a comprehensive, filterable view of all non-Draft `ToolRequest` activity.

- **Date Range Filter:** `from_date` and `to_date` GET parameters are parsed with Django's `parse_date()` utility and applied as `__date__gte` / `__date__lte` queryset filters.
- **Client-Side Search:** A JavaScript search bar on `master_report.html` filters the visible table rows in real time by Tool Code without a page reload.

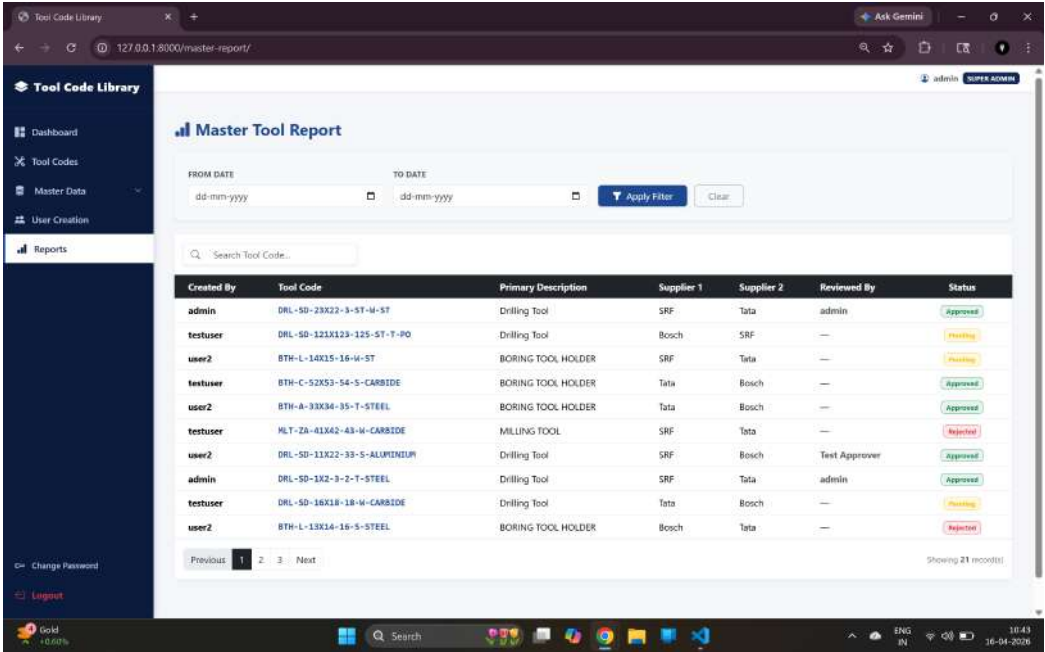


Figure 4.10: Master Report Interface with date-range filters and real-time search

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

The development of the Tool Code Library System has been a journey from identifying a critical gap in manual tool code governance to delivering a fully functional, auditable, and automated enterprise solution. By leveraging Django’s MVT architecture and a carefully designed relational schema, the system delivers a platform that does not merely store data — it actively enforces structure, traceability, and consistency across the entire tool code lifecycle.

The project successfully balances technical rigour with operational simplicity. While the backend enforces strict permission boundaries, handles complex workflow state transitions, and runs a collision-safe code generation algorithm, the frontend provides an intuitive, dynamically adaptive experience through the AJAX-powered form renderer and role-specific dashboard views.

Three architectural decisions stand out as the most impactful contributions of this project:

- The **dynamic RBAC model** — driven entirely by database boolean flags — provides unprecedented organisational flexibility, allowing access policies to be redefined at runtime without any engineering involvement.
- The **Soft Delete pattern**, implemented once in `SoftDeleteModel` and inherited by all master data models, guarantees that no historical reference data is ever lost, even as the operational catalog is continuously updated.
- The **AuditLog model**, capturing every authentication event with actor identity, IP address, and HTTP metadata, provides a solid foundation for security monitoring and organisational compliance reporting.

Ultimately, this system delivers more than just unique code identifiers; it delivers reliability, accountability, and peace of mind by ensuring that every tool code is validated, uniquely generated, and permanently traceable to the person who created and approved it.

5.2 Future Scope and Enhancements

While the current version of the system is robust and fully operational, there is a clear path for it to evolve into an even more powerful enterprise platform:

- **Email Notification Engine:** Integrate Django's email backend to send automated alerts to Requesters when their submissions are approved or rejected, and to notify Approvers when new requests enter the Pending queue.
- **Bulk CSV / Excel Import:** A dedicated import view that reads a structured CSV and creates multiple `ToolRequest` records in a single transaction, enabling migration of legacy tool code data without manual re-entry.
- **Report Export (Excel / PDF):** Add one-click export of the Master Report table to `.xlsx` (via `openpyxl`) and PDF format for offline sharing and management sign-off.
- **File Attachments on Requests:** Add a `FileField` to `ToolRequest` for attaching technical drawings, specification sheets, or purchase orders, with file type and size validation mirroring industry standards.
- **Mobile-Responsive UI:** Refactor all 14 templates with a responsive CSS framework (Bootstrap or Tailwind CSS) to ensure full usability on tablet and mobile devices for on-floor engineers.
- **RESTful API Layer:** Expose tool code data via a documented REST API (Django REST Framework) to enable integration with ERP systems such as SAP or Oracle for real-time inventory synchronisation.
- **Automated Test Suite:** Replace the current manual testing approach with a comprehensive Django `TestCase` suite covering all 25 views, model methods, and the code generation algorithm.

References

- [1] **Django Software Foundation.** (2026). *Django Documentation: The Model-View-Template (MVT) Pattern*. Retrieved from <https://docs.djangoproject.com/>
- [2] **Python Software Foundation.** (2026). *Python 3.x Standard Library: SMTP and Background Execution*. Retrieved from <https://docs.python.org/3/>
- [3] **Microsoft Corporation.** (2026). *Microsoft SQL Server Documentation: Relational Database Management*. Retrieved from <https://learn.microsoft.com/sql/>
- [4] **Agile Alliance.** (2026). *Principles of the Agile Software Development Lifecycle*. Retrieved from <https://www.agilealliance.org/>
- [5] **Ferraiolo, D., & Kuhn, D. R.** (1992). *Role-Based Access Controls*. Proceedings of the 15th National Computer Security Conference, pp. 554–563.
- [6] **Aequus SEZ Private Limited.** (2026). *Internal IT Security Guidelines for Web Applications*.